

Data Structure Lab5 : Circularly Linked List 2022-2023

Topics

1. Implement Node Class
2. Implement CircularlyLinkedList Class
3. Implement Basic Methods of CircularlyLinkedList
 - isEmpty()
 - size()
 - first()
 - last()
 - addFirst()
 - addLast()
 - removeFirst()
 - rotate()

Homework

1. Consider the implementation of CircularlyLinkedList.addFirst, in Code Fragment 3.16. The else body at lines 39 and 40 of that method relies on a locally declared variable, newest. Redesign that clause to avoid use of any local variable.

```
public void addFirst(T data) {  
    if (isEmpty()) {  
        head = new Node<>(data, null);  
        head.next = head;  
    } else {  
        head = new Node<>(data, head.next);  
        tail.next = head;  
    }  
}
```

Data Structure Lab5 : Circularly Linked List 2022-2023

2. Give an implementation of the size() method for the CircularlyLinkedList class, assuming that we did not maintain size as an instance variable.

```
public int size() {  
    if (isEmpty()) return 0;  
  
    int count = 1;  
    Node<T> current = head.next;  
    while (current != head) {  
        count++;  
        current = current.next;  
    }  
    return count;  
}
```

3. Implement the equals() method for the CircularlyLinkedList class, assuming that two lists are equal if they have the same sequence of elements, with corresponding elements currently at the front of the list.

```
public boolean equals(Object obj) {  
    if (this == obj) return true;  
    if (!(obj instanceof CircularlyLinkedList<?>)) return false;  
  
    CircularlyLinkedList<?> other = (CircularlyLinkedList<?>) obj;  
    if (this.size() != other.size()) return false;  
  
    Node<T> currentA = this.head;  
    Node<?> currentB = other.head;  
    do {  
        if (!currentA.data.equals(currentB.data)) return false;  
        currentA = currentA.next;  
        currentB = currentB.next;  
    } while (currentA != this.head);  
}
```

Data Structure Lab5 : Circularly Linked List 2022-2023

```
    return true;
}
```

4. Suppose you are given two circularly linked lists, L and M. Describe an algorithm for telling if L and M store the same sequence of elements (but perhaps with different starting points).

```
public boolean sameSequence(CircularlyLinkedList<T> other) {
    if (this.size() != other.size()) return false;
    if (this.isEmpty()) return true;
```

```
    Node<T> currentL = this.head;
    Node<?> currentM = other.head;
```

```
    do {
        Node<?> tempM = currentM;
        do {
            if (currentL.data.equals(tempM.data)) {
                Node<T> tempL = currentL;
                while (tempL != currentL) {
                    if (!tempL.data.equals(tempM.data)) {
                        return false;
                    }
                    tempL = tempL.next;
                    tempM = tempM.next;
                }
                return true;
            }
            tempM = tempM.next;
        } while (tempM != other.head);
        currentL = currentL.next;
    } while (currentL != this.head);
```

Data Structure Lab5 : Circularly Linked List 2022-2023

```
        return false;
    }
5. Given a circularly linked list L containing an even number of nodes,
   describe how to split L into two circularly linked lists of half the size.
   public void split(CircularlyLinkedList<T> list1, CircularlyLinkedList<T>
   list2) {
       if (isEmpty() || size() % 2 != 0) {
           throw new IllegalArgumentException("List must have an even
           number of nodes");
       }

       Node<T> slow = head;
       Node<T> fast = head;

       while (fast.next != head && fast.next.next != head) {
           slow = slow.next;
           fast = fast.next.next;
       }

       list1.head = head;
       list2.head = slow.next;
       slow.next = head;
       fast.next = list2.head;
   }
6. Implement the clone( ) method for the CircularlyLinkedList class.
   public CircularlyLinkedList<T> clone() {
       CircularlyLinkedList<T> cloneList = new CircularlyLinkedList<>();

       if (isEmpty()) return cloneList;

       Node<T> current = head;
       do {
```

Data Structure Lab5 : Circularly Linked List 2022-2023

```
        cloneList.addLast(current.data);
        current = current.next;
    } while (current != head);

    return cloneList;
}
```